

ACTIVIDAD DE APRENDIZAJE 2

Rey David Borrego Pérez

7502410022

Desarrollo de Software Web

Ingeniería de Software

Universidad de Cartagena

2026-1



Resumen

El presente trabajo describe el proceso de diseño e implementación de una aplicación web desarrollada con Spring Boot MVC, Thymeleaf, Spring Data JPA y base de datos H2. El proyecto toma como punto de partida las guías de desarrollo básico de aplicaciones web con Spring Boot estructuradas bajo el patrón Modelo-Vista-Controlador, y las adapta a un caso académico compuesto por la gestión de usuarios y la gestión de pedidos de restaurante. La solución implementa operaciones CRUD completas, formularios HTML, validaciones de datos, mensajes de retroalimentación y separación de responsabilidades entre controladores, servicios, repositorios, modelos y vistas. El control de versiones fue planteado mediante Git y GitHub, con una estrategia de ramas por funcionalidad, commits progresivos y documentación en el archivo README. Los resultados evidencian la importancia del patrón MVC como base para construir aplicaciones web organizadas, mantenibles y comprensibles dentro del proceso formativo en ingeniería de software.

1. Introducción

El desarrollo de aplicaciones web exige una organización clara del código fuente, especialmente cuando el sistema debe gestionar información, validar entradas del usuario y presentar resultados a través de una interfaz gráfica. En este contexto, el patrón Modelo-Vista-Controlador constituye una alternativa adecuada para separar la lógica de presentación, la lógica de negocio y el acceso a los datos, permitiendo que cada componente del sistema tenga una responsabilidad definida.

El presente proyecto fue desarrollado en el marco de la asignatura Desarrollo de Software Web, correspondiente al programa de Ingeniería de Software de la Universidad de Cartagena. Su propósito central es construir una aplicación web funcional basada en Spring Boot MVC que implemente las operaciones principales de un CRUD para usuarios y para pedidos de restaurante, tomando como referencia las guías de trabajo propuestas en la unidad.

La actividad permite aplicar conceptos fundamentales del desarrollo web moderno en Java, entre ellos la construcción de controladores, el uso de plantillas Thymeleaf, la persistencia de datos mediante Spring Data JPA, la definición de entidades y repositorios, y la validación de formularios. Además, el ejercicio fortalece el uso de Git y GitHub como herramientas de

trazabilidad, dado que el proceso debe evidenciarse mediante commits progresivos y ramas de trabajo.

Repositorio del proyecto: <https://github.com/ReyDavid07/unidad2-spring-mvc-crud.git>

2. Marco Teórico

2.1. Patrón Modelo-Vista-Controlador (MVC)

El patrón Modelo-Vista-Controlador, conocido como MVC, es una estrategia de diseño ampliamente utilizada en aplicaciones web para separar las responsabilidades del sistema. El modelo representa los datos y las reglas asociadas a ellos; la vista se encarga de presentar la información al usuario; y el controlador recibe las solicitudes, coordina las operaciones necesarias y define la respuesta que será mostrada. Esta separación mejora la mantenibilidad porque permite modificar una parte del sistema sin afectar innecesariamente las demás.

En el proyecto desarrollado, el patrón MVC se evidencia en la organización de paquetes. Las clases de modelo representan las entidades Usuario y Pedido; los controladores gestionan las rutas web y las acciones del usuario; los servicios concentran la lógica de aplicación; y las plantillas HTML construidas con Thymeleaf constituyen la capa de vista.

2.2. Spring Boot y desarrollo web en Java

Spring Boot es un framework que simplifica la creación de aplicaciones Java al ofrecer configuración automática, servidor embebido y una estructura de dependencias lista para el desarrollo web. En este proyecto se utilizó Spring Boot como base tecnológica para crear una aplicación MVC que pudiera ejecutarse localmente, atender peticiones HTTP y conectar las vistas con la lógica del sistema.

El uso de Spring Boot facilita el proceso académico porque reduce la configuración inicial y permite concentrar el trabajo en la construcción de funcionalidades. A través de anotaciones como `@Controller`, `@Service`, `@Repository` y `@Entity`, el framework permite identificar la función de cada clase dentro de la aplicación y administrar sus dependencias mediante inversión de control.

2.3. Persistencia con Spring Data JPA y ORM

Spring Data JPA permite trabajar con persistencia de datos a partir de repositorios que abstraen las operaciones más comunes sobre la base de datos. En lugar de escribir manualmente todas las consultas SQL, el desarrollador define interfaces que heredan de `JpaRepository` y obtiene métodos para guardar, listar, buscar, actualizar y eliminar registros.

El mapeo objeto-relacional, conocido como ORM, permite representar tablas de una base de datos mediante clases Java. En esta actividad, las entidades Usuario y Pedido se mapearon como clases persistentes, lo cual permitió almacenar los datos en una base H2 en memoria durante la ejecución del sistema. Esta elección resulta adecuada para pruebas académicas porque no exige instalar un gestor externo de base de datos.

2.4. Thymeleaf y capa de presentación

Thymeleaf es un motor de plantillas que permite construir páginas HTML dinámicas integradas con aplicaciones Spring Boot. A través de atributos como `th:each`, `th:object`, `th:field` y `th:if`, las vistas pueden recibir información desde los controladores, recorrer listas de datos, llenar formularios y mostrar mensajes de error o confirmación.

En la solución de la Unidad 2, Thymeleaf se empleó para construir la página de inicio, los listados de usuarios y pedidos, y los formularios de creación y edición. De esta manera, el usuario puede interactuar con el sistema desde una interfaz web sin depender de herramientas externas.

2.5. Control de Versiones con Git y GitHub

Git es un sistema de control de versiones distribuido que permite registrar la evolución del código fuente mediante commits. GitHub, por su parte, funciona como plataforma remota para alojar repositorios, revisar cambios y evidenciar el proceso de desarrollo. En esta actividad, el uso de Git y GitHub es fundamental porque el tutor solicita trabajar con ramas, commits progresivos y envíos organizados al repositorio.

3. Objetivos

3.1. Objetivo General

Desarrollar una aplicación web funcional con Spring Boot MVC que permita gestionar usuarios y pedidos de restaurante mediante operaciones CRUD, aplicando separación de responsabilidades, persistencia con JPA, validación de formularios y control de versiones con Git y GitHub.

3.2. Objetivos Específicos

- Implementar una estructura de proyecto basada en el patrón Modelo-Vista-Controlador para organizar la aplicación de forma clara y mantenible.
- Crear un módulo de gestión de usuarios que permita listar, registrar, editar y eliminar información desde una interfaz web.
- Desarrollar un módulo de gestión de pedidos de restaurante como caso asignado, incorporando campos propios del número de pedido, cliente, producto, cantidad, precio y estado.
- Aplicar validaciones en los formularios para evitar el registro de información incompleta o incorrecta.
- Utilizar Spring Data JPA y H2 para almacenar y consultar los datos durante la ejecución de la aplicación.
- Documentar el proceso de desarrollo mediante Git, GitHub, README, commits progresivos y ramas de funcionalidad.

4. Tecnologías Utilizadas

La selección de tecnologías para esta actividad respondió a los requerimientos de la unidad y a la necesidad de construir una aplicación web funcional, sencilla de ejecutar y organizada por capas. A continuación se describen las herramientas principales utilizadas.

4.1. Java y Spring Boot

Java fue empleado como lenguaje de programación principal por su madurez, orientación a objetos y compatibilidad con el ecosistema Spring. Spring Boot permitió crear el proyecto,

configurar el servidor embebido y administrar las dependencias necesarias para el desarrollo MVC.

4.2. Thymeleaf, HTML y CSS

Thymeleaf fue utilizado como motor de plantillas para crear vistas dinámicas integradas con Spring. HTML permitió estructurar los formularios y tablas, mientras que CSS se empleó para mejorar la presentación visual de la aplicación y facilitar la navegación del usuario.

4.3. Spring Data JPA y H2

Spring Data JPA permitió implementar repositorios para acceder a los datos de manera sencilla, mientras que H2 se utilizó como base de datos en memoria para realizar pruebas rápidas sin instalar un motor externo. Esta combinación resulta adecuada para una entrega académica porque simplifica la ejecución del proyecto en diferentes equipos.

4.4. Maven, Git y GitHub

Maven se empleó para gestionar dependencias y ejecutar la aplicación. Git permitió controlar las versiones del código, y GitHub funcionó como repositorio remoto para evidenciar el historial de commits, la organización por ramas y la documentación del proyecto.

5. Desarrollo del Sistema

5.1. Estructura General del Proyecto

La aplicación se organizó en paquetes que representan las responsabilidades principales del patrón MVC. La estructura base del proyecto incluye controladores, modelos, repositorios, servicios, recursos estáticos y plantillas HTML. Esta organización permite ubicar rápidamente cada componente y facilita la ampliación futura del sistema.

Componente	Ubicación	Responsabilidad
Controladores	controller	Reciben solicitudes web, cargan datos del modelo y retornan las vistas correspondientes.
Modelos	model	Representan las entidades Usuario y Pedido con sus atributos y validaciones.
Repositorios	repository	Abstraen el acceso a la base de datos mediante Spring Data JPA.

Servicios	service	Centralizan la lógica de aplicación y coordinan las operaciones CRUD.
Vistas	templates	Presentan formularios, listados y páginas HTML mediante Thymeleaf.
Estilos	static/css	Definen la presentación visual de la interfaz.

5.2. Módulo de Gestión de Usuarios

El módulo de usuarios corresponde al CRUD base trabajado en las guías. Este módulo permite visualizar los usuarios registrados, crear nuevos usuarios, actualizar sus datos y eliminar registros. La entidad Usuario contempla información básica como nombre, correo electrónico, contraseña, rol y estado, lo que permite simular una administración sencilla de usuarios dentro del sistema.

Desde la interfaz, el usuario puede acceder al listado, seleccionar la opción de registrar un nuevo usuario o editar uno existente. Los datos ingresados pasan por validaciones básicas antes de ser enviados al servicio. Posteriormente, el repositorio se encarga de persistir la información en la base de datos H2.

5.3. Módulo de Gestión de Pedidos de Restaurante

El segundo módulo corresponde al caso asignado de restaurante, centrado en la entidad Pedido. Este componente permite administrar información relacionada con el número de pedido, el nombre del cliente, la descripción del producto o plato solicitado, la cantidad, el precio total y el estado del pedido. De esta manera, el sistema no solo reproduce el ejemplo de usuarios, sino que lo adapta a una nueva tabla con lógica propia del contexto planteado.

El CRUD de pedidos mantiene la misma estructura del módulo de usuarios, lo cual demuestra reutilización del modelo de trabajo propuesto en las guías. La aplicación permite registrar pedidos, consultarlos en una tabla, modificar sus datos cuando sea necesario y eliminarlos mediante una acción confirmada por el usuario.

5.4. Rutas y funcionalidades implementadas

Ruta	Funcionalidad	Descripción
/	Inicio	Muestra la página principal con acceso a usuarios y

		pedidos.
/usuarios	Listar usuarios	Presenta todos los usuarios registrados en el sistema.
/usuarios/nuevo	Crear usuario	Carga el formulario para registrar un usuario nuevo.
/usuarios/editar/{id}	Editar usuario	Permite modificar datos de un usuario existente.
/usuarios/eliminar/{id}	Eliminar usuario	Elimina un usuario seleccionado.
/pedidos	Listar pedidos	Presenta todos los pedidos registrados.
/pedidos/nuevo	Crear pedido	Carga el formulario para registrar un pedido nuevo.
/pedidos/editar/{id}	Editar pedido	Permite modificar los datos de un pedido existente.
/pedidos/eliminar/{id}	Eliminar pedido	Elimina el pedido seleccionado.

5.5. Validaciones y mensajes de retroalimentación

Los formularios de usuarios y pedidos incorporan validaciones para controlar que la información ingresada cumpla con los requisitos mínimos del sistema. Estas validaciones permiten evitar registros incompletos y mejoran la experiencia del usuario porque los errores pueden mostrarse directamente en la vista antes de finalizar la operación.

Además, la aplicación contempla mensajes de confirmación y navegación clara entre páginas, lo cual facilita el uso del sistema. La separación de plantillas para listados y formularios permite mantener una interfaz comprensible y coherente con las operaciones CRUD esperadas.

6. Pruebas Realizadas

La verificación del funcionamiento se planteó mediante pruebas funcionales manuales. Aunque el entorno de generación del informe no permite ejecutar Maven ni levantar la aplicación localmente, el proyecto queda estructurado para ser probado en un equipo con Java 17 y Maven instalados. Las pruebas deben realizarse ejecutando el comando `mvn spring-boot:run` y accediendo a las rutas principales desde el navegador.

- Acceso a la página principal del sistema y navegación hacia los módulos de usuarios y pedidos.
- Registro de usuarios con datos válidos y validación ante campos incompletos.
- Listado de usuarios cargados desde la base de datos H2.
- Actualización de información de un usuario existente.
- Eliminación de usuarios desde la tabla de administración.

- Registro de pedidos con número de pedido, cliente, producto, cantidad, precio y estado.
- Consulta del listado de pedidos después de crear, actualizar y eliminar registros.
- Verificación de la consola H2 para comprobar la persistencia de los datos durante la ejecución.

Los escenarios anteriores permiten comprobar que el sistema cumple con las operaciones esperadas para una aplicación CRUD MVC. Como mejora futura, se recomienda incorporar pruebas automatizadas de controladores y servicios para complementar las pruebas manuales.

7. Control de Versiones

El ciclo de vida del desarrollo debe gestionarse con Git y GitHub, siguiendo una estrategia de commits pequeños y descriptivos. Para cumplir con la orientación del tutor, el repositorio no debe subirse en un único commit final, sino mediante avances progresivos que evidencien la construcción del proyecto.

- Rama principal (main): contiene la versión estable del proyecto, lista para ser revisada por el tutor.
- Ramas de funcionalidad (feature/): se recomienda utilizar ramas como feature/estructura-base, feature/crud-usuarios, feature/crud-pedidos y feature/vistas-css.
- Commits progresivos: cada commit debe representar un avance pequeño, por ejemplo la creación del proyecto base, la implementación del modelo, la creación de repositorios, la construcción de controladores, la elaboración de vistas o la documentación del README.
- Push organizado: los cambios deben enviarse al repositorio remoto de manera progresiva y con mensajes claros, evitando subir archivos generados como target o configuraciones innecesarias del entorno local.

Esta estrategia permite demostrar el proceso real de construcción del sistema y facilita la revisión del historial en GitHub.

8. Conclusiones

El desarrollo de la actividad permitió comprender de manera práctica la utilidad del patrón MVC para organizar aplicaciones web. Al separar controladores, modelos, servicios, repositorios y

vistas, el sistema se vuelve más claro, más fácil de mantener y más adecuado para incorporar nuevas funcionalidades.

En segundo lugar, el uso de Spring Boot facilitó la creación de una aplicación funcional al integrar servidor embebido, gestión de dependencias y configuración automática. Esto permitió concentrar el esfuerzo en la implementación del CRUD y en la adaptación del ejemplo base al caso de pedidos de restaurante.

En tercer lugar, la integración de Thymeleaf, Spring Data JPA y H2 permitió desarrollar una solución completa que combina interfaz web, persistencia de datos y validación de formularios. Esta combinación tecnológica resulta apropiada para proyectos académicos porque permite evidenciar el ciclo completo de una aplicación web sin requerir infraestructura compleja.

Finalmente, el uso de Git y GitHub fortalece las buenas prácticas de desarrollo, ya que permite registrar el avance del proyecto mediante commits, trabajar con ramas y presentar un historial claro del proceso. Como trabajo futuro, se propone implementar autenticación, paginación de listados, pruebas automatizadas y despliegue en un entorno web.

Referencias

Chacon, S., y Straub, B. (2014). Pro Git (2.a ed.). Apress. <https://git-scm.com/book/en/v2>

Fowler, M. (2002). Patterns of enterprise application architecture. Addison-Wesley Professional.

Oracle. (2024). Java documentation. <https://docs.oracle.com/en/java/>

Spring. (2024). Spring Boot reference documentation. <https://docs.spring.io/spring-boot/docs/current/reference/html/>

Spring. (2024). Spring Data JPA reference documentation. <https://docs.spring.io/spring-data/jpa/reference/>

Thymeleaf. (2024). Thymeleaf documentation. <https://www.thymeleaf.org/documentation.html>